```
bool isNumber(string s) {
    if (s.empty()) return false;

    for (int i = 0; i < (int)s.length(); i++) {
        if (!isdigit((unsigned char)s[i]))
            return false;
    }
    return true;
}

void trim(string &s) {
    size_t a = s.find_first_not_of(" \t");
    size_t b = s.find_last_not_of(" \t");

    if (a == string::npos)
        s = "";
    else
        s = s.substr(a, b - a + 1);
}

void lexicalAnalysis(string s) {
    tokens.clear();

    for (int i = 0; i < (int)s.length();) {
        if (isspace((unsigned char)s[i])) {
            i++;
        }
        else if (isalpha((unsigned char)s[i]) || s[i] == '_') {
            string word = "";

            while (i < (int)s.length() &&
                   (isalnum((unsigned char)s[i]) || s[i] == '_')) {
                word += s[i];
                i++;
            }
```

```cpp
#include <iostream>
#include <sstream>
#include <vector>
#include <map>
#include <set>
#include <string>
#include <cctype>
#include <stdexcept>
using namespace std;

struct Token {
    string value;
    string type;
};

struct Node {
    string value;
    Node *left;
    Node *right;

    Node(string v) {
        value = v;
        left = NULL;
        right = NULL;
    }
};

vector<Token> tokens;
vector<string> tac;
vector<string> optimized;
set<string> declared;
int pos = 0;
int tempCount = 0;
```

```
====================================
           MINI COMPILER
====================================
Enter statements (empty line to finish):
a = 10;
b = 20;
c = a + b * 2;
d = 5 * 4;


====================================
Compiling: a = 10

--- LEXICAL ANALYSIS ---
10 : Number

--- SYNTAX ANALYSIS ---
Syntax is correct.

--- SEMANTIC ANALYSIS ---
Semantic analysis successful.

--- INTERMEDIATE CODE GENERATION ---
a = 10

--- CODE OPTIMIZATION ---
Optimized Three Address Code:
a = 10

--- TARGET CODE GENERATION ---
```

```
Compilation successful!

==================================
Compiling: d = 5 * 4

--- LEXICAL ANALYSIS ---
5 : Number
* : Operator
4 : Number

--- SYNTAX ANALYSIS ---
Syntax is correct.

--- SEMANTIC ANALYSIS ---
Semantic analysis successful.

--- INTERMEDIATE CODE GENERATION ---
t1 = 5 * 4
d = t1

--- CODE OPTIMIZATION ---
Optimized Three Address Code:
d = 20

--- TARGET CODE GENERATION ---
MOV d, 20

Compilation successful!
```

```
Compiling: c = a + b * 2

--- LEXICAL ANALYSIS ---
a : Identifier
+ : Operator
b : Identifier
* : Operator
2 : Number

--- SYNTAX ANALYSIS ---
Syntax is correct.

--- SEMANTIC ANALYSIS ---
Semantic analysis successful.

--- INTERMEDIATE CODE GENERATION ---
t1 = b * 2
t2 = a + t1
c = t2

--- CODE OPTIMIZATION ---
Optimized Three Address Code:
t1 = b * 2
t2 = a + t1
c = t2

--- TARGET CODE GENERATION ---
MOV R1, b
MUL R1, 2
MOV t1, R1
```

```
--- TARGET CODE GENERATION ---
MOV a, 10

Compilation successful!

====================================
Compiling: b = 20

--- LEXICAL ANALYSIS ---
20 : Number

--- SYNTAX ANALYSIS ---
Syntax is correct.

--- SEMANTIC ANALYSIS ---
Semantic analysis successful.

--- INTERMEDIATE CODE GENERATION ---
b = 20

--- CODE OPTIMIZATION ---
Optimized Three Address Code:
b = 20

--- TARGET CODE GENERATION ---
MOV b, 20

Compilation successful!

====================================
```

```
#include <iostream>
#include <sstream>
#include <vector>
#include <map>
#include <set>
#include <string>
#include <cctype>
#include <stdexcept>
using namespace std;
```

```
struct Token {
   string value;
   string type;
};

struct Node {
   string value;
   Node *left;
   Node *right;

   Node(string v) {
      value = v;
      left = NULL;
      right = NULL;
   }
};

vector<Token> tokens;
vector<string> tac;
vector<string> optimized;
set<string> declared;
int pos = 0;
int tempCount = 0;

string numberString(int n) {
   stringstream ss;
   ss << n;
   return ss.str();
}
```

```
int toNumber(string s) {
   stringstream ss(s);
   int n;
   ss >> n;
   return n;
}

bool isNumber(string s) {
   if (s.empty()) return false;

   for (int i = ٠; i < (int)s.length(); i++) {
      if (!isdigit((unsigned char)s[i]))
         return false;
   }
   return true;
}

void trim(string &s) {
   size_t a = s.find_first_not_of(" \t");
   size_t b = s.find_last_not_of(" \t");

   if (a == string::npos)
      s = "";
   else
      s = s.substr(a, b - a + ١);
}

void lexicalAnalysis(string s) {
   tokens.clear();
```

```
for (int i = 0; i < (int)s.length();) {
    if (isspace((unsigned char)s[i])) {
        i++;
    }
    else if (isalpha((unsigned char)s[i]) || s[i] == '_') {
        string word = "";

        while (i < (int)s.length() &&
               (isalnum((unsigned char)s[i]) || s[i] == '_')) {
            word += s[i];
            i++;
        }

        Token t;
        t.value = word;
        t.type = "Identifier";
        tokens.push_back(t);
    }
    else if (isdigit((unsigned char)s[i])) {
        string num = "";

        while (i < (int)s.length() &&
               isdigit((unsigned char)s[i])) {
            num += s[i];
            i++;
        }

        Token t;
        t.value = num;
        t.type = "Number";
```

```
            tokens.push_back(t);
        }
        else if (string("+-*/").find(s[i]) != string::npos) {
            Token t;
            t.value = string(1, s[i]);
            t.type = "Operator";
            tokens.push_back(t);
            i++;
        }
        else if (s[i] == '(' || s[i] == ')') {
            Token t;
            t.value = string(1, s[i]);
            t.type = "Parenthesis";
            tokens.push_back(t);
            i++;
        }
        else {
            throw runtime_error("Invalid character found!");
        }
    }

    cout << "\n--- LEXICAL ANALYSIS ---\n";

    for (int i = 0; i < (int)tokens.size(); i++) {
        cout << tokens[i].value << " : "
             << tokens[i].type << endl;
    }
}

Node* expression();
```

```
Node* factor() {
    if (pos >= (int)tokens.size())
        throw runtime_error("Unexpected end of expression!");

    string v = tokens[pos].value;

    if (tokens[pos].type == "Number" ||
        tokens[pos].type == "Identifier") {
        pos++;
        return new Node(v);
    }

    if (v == "(") {
        pos++;
        Node *n = expression();

        if (pos >= (int)tokens.size() ||
            tokens[pos].value != ")")
            throw runtime_error("Missing closing parenthesis!");

        pos++;
        return n;
    }

    throw runtime_error("Invalid expression!");
}

Node* term() {
    Node *n = factor();
```

```
    while (pos < (int)tokens.size() &&
           (tokens[pos].value == "*" ||
            tokens[pos].value == "/")) {

        string op = tokens[pos].value;
        pos++;

        Node *r = factor();
        Node *temp = new Node(op);

        temp->left = n;
        temp->right = r;
        n = temp;
    }

    return n;
}

Node* expression() {
    Node *n = term();

    while (pos < (int)tokens.size() &&
           (tokens[pos].value == "+" ||
            tokens[pos].value == "-")) {

        string op = tokens[pos].value;
        pos++;

        Node *r = term();
```

```
        Node *temp = new Node(op);

        temp->left = n;
        temp->right = r;
        n = temp;
    }

    return n;
}

void syntaxAnalysis() {
    if (pos != (int)tokens.size())
        throw runtime_error("Unexpected token in expression!");

    cout << "\n--- SYNTAX ANALYSIS ---\n";
    cout << "Syntax is correct.\n";
}

void semanticAnalysis(Node *n) {
    if (n == NULL) return;

    if (n->left == NULL && n->right == NULL) {
        if (!isNumber(n->value) &&
            declared.find(n->value) == declared.end()) {
            throw runtime_error("Undefined variable: " + n->value);
        }
        return;
    }

    semanticAnalysis(n->left);
```

```
    semanticAnalysis(n->right);

    if (n->value == "/" &&
        n->right->left == NULL &&
        n->right->right == NULL &&
        isNumber(n->right->value) &&
        toNumber(n->right->value) == 0) {
        throw runtime_error("Division by zero!");
    }
}

string generateTAC(Node *n) {
    if (n->left == NULL && n->right == NULL)
        return n->value;

    string a = generateTAC(n->left);
    string b = generateTAC(n->right);

    string temp = "t" + numberString(++tempCount);

    tac.push_back(temp + " = " + a + " " +
                n->value + " " + b);

    return temp;
}

void intermediateCode(Node *root, string lhs) {
    tac.clear();
    tempCount = 0;
```

```
    string result = generateTAC(root);

    tac.push_back(lhs + " = " + result);

    cout << "\n--- INTERMEDIATE CODE GENERATION ---\n";

    for (int i = 0; i < (int)tac.size(); i++)
        cout << tac[i] << endl;
}

void codeOptimization() {
    cout << "\n--- CODE OPTIMIZATION ---\n";

    optimized.clear();

    map<string, string> constants;

    for (int i = 0; i < (int)tac.size(); i++) {
        string result, equal, a, op, b;
        stringstream ss(tac[i]);

        ss >> result >> equal >> a;

        bool operation = false;

        if (ss >> op >> b)
            operation = true;

        if (constants.find(a) != constants.end())
            a = constants[a];
```

```
if (operation &&
    constants.find(b) != constants.end())
    b = constants[b];

if (operation && isNumber(a) && isNumber(b)) {
    int x = toNumber(a);
    int y = toNumber(b);
    int value = 0;

    if (op == "+") value = x + y;
    else if (op == "-") value = x - y;
    else if (op == "*") value = x * y;
    else if (op == "/") {
        if (y == 0)
            throw runtime_error("Division by zero!");

        value = x / y;
    }

    string folded = numberString(value);
    constants[result] = folded;

    optimized.push_back(result + " = " + folded);
}
else if (operation) {
    constants.erase(result);

    optimized.push_back(
        result + " = " + a + " " + op + " " + b
```

```
        );
    }
    else {
        if (isNumber(a))
            constants[result] = a;
        else
            constants.erase(result);

        optimized.push_back(result + " = " + a);
    }
}

set<string> needed;
vector<string> finalCode;

for (int i = (int)optimized.size() - ১; i >= ০; i--) {
    string result, equal, a, op, b;
    stringstream ss(optimized[i]);

    ss >> result >> equal >> a;

    bool operation = false;

    if (ss >> op >> b)
        operation = true;

    bool keep = true;

    if (result.length() > ১ &&
        result[০] == 't' &&
```

```
            isdigit((unsigned char)result[১])) {

            if (needed.find(result) == needed.end())
                keep = false;
        }

        if (keep) {
            finalCode.insert(finalCode.begin(), optimized[i]);

            if (operation) {
                if (!isNumber(a)) needed.insert(a);
                if (!isNumber(b)) needed.insert(b);
            }
            else if (!isNumber(a)) {
                needed.insert(a);
            }
        }
    }

    optimized = finalCode;

    cout << "Optimized Three Address Code:\n";

    for (int i = ০; i < (int)optimized.size(); i++)
        cout << optimized[i] << endl;
}

void targetCodeGeneration() {
    cout << "\n--- TARGET CODE GENERATION ---\n";
```

```
    for (int i = 0; i < (int)optimized.size(); i++) {
        string result, equal, a, op, b;
        stringstream ss(optimized[i]);

        ss >> result >> equal >> a;

        if (!(ss >> op >> b)) {
            cout << "MOV " << result << ", " << a << endl;
        }
        else {
            cout << "MOV R3, " << a << endl;

            if (op == "+")
                cout << "ADD R3, " << b << endl;
            else if (op == "-")
                cout << "SUB R3, " << b << endl;
            else if (op == "*")
                cout << "MUL R3, " << b << endl;
            else if (op == "/")
                cout << "DIV R3, " << b << endl;

            cout << "MOV " << result << ", R3" << endl;
        }
    }
}

int main() {
    string line;
    vector<string> statements;
```

```
cout << "====================================\n";
cout << "          MINI COMPILER\n";
cout << "====================================\n";

cout << "Enter statements (empty line to finish):\n";

while (getline(cin, line)) {
    trim(line);

    if (line.empty())
        break;

    statements.push_back(line);
}

try {
    for (int k = ০; k < (int)statements.size(); k++) {
        string s = statements[k];

        if (s.length() == ০ || s[s.length() - ১] != ';')
            throw runtime_error("Missing semicolon!");

        s = s.substr(০, s.length() - ১);

        size_t p = s.find('=');

        if (p == string::npos ||
            s.find('=', p + ১) != string::npos)
            throw runtime_error("Invalid assignment statement!");
```

```
string lhs = s.substr(0, p);
string rhs = s.substr(p + 1);

trim(lhs);
trim(rhs);

if (lhs.empty() || rhs.empty())
   throw runtime_error("Invalid assignment!");

if (!(isalpha((unsigned char)lhs[0]) ||
     lhs[0] == '_'))
   throw runtime_error("Invalid variable name!");

for (int i = 0; i < (int)lhs.length(); i++) {
   if (!(isalnum((unsigned char)lhs[i]) ||
        lhs[i] == '_'))
      throw runtime_error("Invalid variable name!");
}

cout << "\n=====================================\n";
cout << "Compiling: " << lhs << " = " << rhs << endl;

lexicalAnalysis(rhs);

pos = 0;
Node *root = expression();

syntaxAnalysis();

semanticAnalysis(root);
```

```
        cout << "\n--- SEMANTIC ANALYSIS ---\n";
        cout << "Semantic analysis successful.\n";

        intermediateCode(root, lhs);

        codeOptimization();

        targetCodeGeneration();

        declared.insert(lhs);

        cout << "\nCompilation successful!\n";
      }
    }
    catch (exception &e) {
      cout << "\n*** COMPILATION ERROR ***\n";
      cout << e.what() << endl;
      cout << "Compilation terminated.\n";
      return 1;
    }

    cout << "\n====================================\n";
    cout << "       COMPILATION FINISHED\n";
    cout << "====================================\n";

    return 0;
}
```